my-agents-cli-mod

This project is generated via `agents-cli create my-agents-cli-mod -y` and modified in `./app/agent.py` to create a real-time weather and world clock AI assistant.

Project Structure Overview

```
my-agents-cli-mod/
├── app/         # Core agent application code
│   ├── agent.py               # Main agent logic (customized with real-time APIs)
│   ├── fast_api_app.py        # FastAPI backend server
│   └── app_utils/             # Application utilities and helper functions
├── tests/                     # Unit, integration, and evaluation test suites
├── GEMINI.md                  # AI-assisted development guide
└── pyproject.toml             # Project dependency configuration
```

> 💡 **Tip:** Use Antigravity CLI for AI-assisted development. Project context is pre-configured in `GEMINI.md`.

Requirements

Before starting, ensure you have the following tools installed:

- **uv**: Fast Python package manager (used for all dependency management) - Installation Guide (add packages with `uv add <package>`)
- **agents-cli**: Google Agents CLI - Install via `uv tool install google-agents-cli`
- **Google Cloud SDK**: SDK for GCP services - Installation Guide

---

Quick Start

If `agents-cli` and related skills are not already installed, set them up:

```
uvx google-agents-cli setup
```

Install required packages:

```
agents-cli install
```

Test the agent in a local web server environment:

```
agents-cli playground
```

You can also leverage ADK CLI features using `uv run adk` .

Commands

Command	Description
`agents-cli install`	Install dependency packages using uv
`agents-cli playground`	Launch local interactive development UI
`agents-cli lint`	Run code quality and style checks
`agents-cli eval`	Evaluate agent behavior (generate, score, analyze — see `agents-cli eval --help`)
`uv run pytest tests/unit tests/integration`	Run unit and integration tests
`agents-cli deploy`	Deploy agent to Google Cloud Agent Runtime
`agents-cli publish gemini`	Register deployed agent with Gemini
A2A Inspector	Launch A2A (Agent-to-Agent) Protocol Inspector

🛠️ Project Management

Command	Action
`agents-cli scaffold enhance`	Adds CI/CD pipelines and Terraform infrastructure code
`agents-cli infra cicd`	One-command setup for entire CI/CD pipeline and infrastructure
`agents-cli scaffold upgrade`	Upgrades template to the latest version while preserving user customizations

Development

Modify the agent logic in `app/agent.py` and test with `agents-cli playground`. Code is automatically hot-reloaded upon saving.

Deployment

```
gcloud config set project <your-project-id>
agents-cli deploy
```

- To add CI/CD and Terraform code, run `agents-cli scaffold enhance`.
- To provision production infrastructure, run `agents-cli infra cicd`.

Observability

Telemetry data export to Cloud Trace, BigQuery, and Cloud Logging is built-in by default.

A2A Inspector

This agent natively supports the A2A Protocol (Agent-to-Agent Protocol). To test interoperability with other agents, use A2A Inspector.
For details, see the A2A Inspector documentation.

📂 `my-agents-cli-mod` Detailed Directory Structure and Dynamic Generation Analysis

A detailed guide explaining the roles of all files and directories in `my-agents-cli-mod`, along with their **creation timing (initial scaffolding vs. dynamic generation vs. user customization)**.

```
my-agents-cli-mod/
├── 📁 app/                          # [Initial] Core agent application code
│   ├── 📄 agent.py                  # ✏️ [Initial / User Customized] Agent core logic and tool
definitions
│   ├── 📄 fast_api_app.py           # 🟢 [Initial] FastAPI server supporting REST / SSE / A2A
│   ├── 📄 __init__.py               # 🟢 [Initial] Python package declaration
│   ├── 📁 app_utils/                # 🟢 [Initial] Agent auxiliary utility modules
│   │   ├── 📄 a2a.py                # 🟢 [Initial] A2A protocol communication & Agent Card
handling
│   │   ├── 📄 reasoning_engine.py   # 🟢 [Initial] Vertex AI Reasoning Engine wrapper
│   │   └── 📄 services.py           # 🟢 [Initial] ADK session and artifact service factory
│   ├── 📁 .adk/                     # ⚡ [Runtime Generated] Internal cache directory created on
local execution
│   │   └── 📄 session.db            # ⚡ [Runtime Generated] SQLite DB storing session and
conversation state
│   └── 📁 __pycache__/              # ⚡ [Compilation Generated] Python interpreter bytecode cache
│
├── 📁 deployment/                   # ⚡ [scaffold enhance] Infrastructure and deployment code
directory
│   └── 📁 terraform/                # ⚡ [scaffold enhance] Terraform HCL code for GCP resource
deployment
│       ├── 📁 shared/               # ⚡ Shared infrastructure modules
│       └── 📁 single-project/       # ⚡ Single-project deployment scripts (main.tf, service.tf,
etc.)
│
├── 📁 tests/                        # 🟢 [Initial] Agent test and evaluation directory
│   ├── 📁 unit/                     # 🟢 [Initial] Unit test directory
│   │   └── 📄 test_dummy.py         # 🟢 [Initial] Basic dummy unit test sample
│   ├── 📁 integration/              # 🟢 [Initial] Server integration test directory
│   │   └── 📄 test_server_e2e.py    # 🟢 [Initial] FastAPI and A2A endpoint E2E test
│   └── 📁 eval/                     # 🟢 [Initial] Agent evaluation pipeline
│       ├── 📄 eval_config.yaml      # 🟢 [Initial] Evaluation metrics and config file
│       ├── 📄 response_quality.py   # 🟢 [Initial] LLM response quality automated scorer
│       └── 📁 datasets/             # 🟢 [Initial] Evaluation query dataset folder
│           └── 📄 basic-dataset.json # 🟢 [Initial] Evaluation queries and expected response data
│
├── 📁 .venv/                        # ⚡ [Package Install] Virtual environment created by `agents-
cli install` / `uv sync`
├── 📁 .pytest_cache/                # ⚡ [Test Cache] Cached pytest execution results
│
├── 📄 agents-cli-manifest.yaml      # 🟢 [Initial] agents-cli metadata (template, agent name,
target runtime)
├── 📄 pyproject.toml                # 🟢 [Initial] uv-based Python project settings and dependency
specification
├── 📄 uv.lock                       # ⚡ [Package Install] Package version lockfile generated on
install
├── 📄 deployment_metadata.json      # ⚡ [Deployment] Remote engine ID and deployment record from
`agents-cli deploy`
├── 📄 Dockerfile                    # 🟢 [Initial] Docker manifest for container builds and
deployment
├── 📄 .env                          # 🟢 [Initial / User Config] API keys, model names,
environment variables
├── 📄 .env.example                  # 🟢 [Initial] Environment configuration template
├── 📄 .gitignore                    # 🟢 [Initial] Git ignore rules
├── 📄 .gcloudignore                 # 🟢 [Initial] GCP deployment upload ignore rules
├── 📄 GEMINI.md                     # 🟢 [Initial] Project guide file for Antigravity/Gemini
coding agents
├── 📄 README.md                     # 🟢 [Initial] Standard English guide
├── 📄 README-KR.md                  # ✏️ [User Added] Detailed Korean documentation and folder
structure guide
└── 📄 README-EN.md                  # 📄 [English Guide] Full English translation of README-KR.md
(this file)
```

📊 File / Directory Classification Criteria

Icon	Classification	Description	Representative Files / Folders
🟢	**Initial Scaffold**	Automatically scaffolded from template upon initial `agents-cli create` command	`app/fast_api_app.py`, `tests/`, `agents-cli-manifest.yaml`
⚡	**Dynamically Generated (Runtime/CLI)**	Created during command execution (`install`, `enhance`, `deploy`, `run`, `pytest`) or during runtime	`.venv/`, `uv.lock`, `deployment/`, `deployment_metadata.json`, `app/.adk/session.db`, `.pytest_cache/`
✏️	**Initial Scaffold & User Customized**	Files initially scaffolded and subsequently edited or added by developers to implement requirements	`app/agent.py`, `.env`, `README-KR.md`, `README-EN.md`